

Multi-Armed Bandit 시뮬레이션

[bandit_avg.py 스크립트 설명서](#)

1. 개요

본 스크립트는 강화학습의 기초 문제인 Multi-Armed Bandit 문제를 epsilon-greedy 전략을 사용하여 해결하는 시뮬레이션입니다. 여러 번의 독립적인 실험을 수행하고 평균 보상을 분석하여 시각화합니다.

2. 주요 개념

2.1 Multi-Armed Bandit 문제

Multi-Armed Bandit은 여러 개의 슬롯 머신(팔) 중에서 어떤 것을 선택해야 최대 보상을 얻을 수 있는지 학습하는 문제입니다. 각 팔은 서로 다른 보상 확률을 가지고 있으며, 에이전트는 시행착오를 통해 최적의 팔을 찾아야 합니다.

2.2 Epsilon-Greedy 전략

Epsilon-greedy는 탐험(Exploration)과 활용(Exploitation)의 균형을 맞추는 전략입니다:

- 활용(Exploitation): 현재까지 가장 좋은 것으로 추정되는 팔을 선택 (확률 $1-\epsilon$)
- 탐험(Exploration): 무작위로 팔을 선택하여 새로운 정보 수집 (확률 ϵ)

본 스크립트에서는 $\epsilon=0.1$ 로 설정하여 90%는 최선의 선택을, 10%는 탐험을 수행합니다.

3. 코드 구조 및 동작 원리

3.1 시뮬레이션 파라미터

```
runs = 200      # ■■■■ ■■ ■■ ■■  
steps = 1000    # ■ ■■■■ ■■ ■  
epsilon = 0.1   # ■■ ■■ (10%)
```

- runs: 통계적 신뢰도를 높이기 위해 200번의 독립적인 실험 수행
- steps: 각 실험마다 1000번의 행동 선택 수행
- epsilon: 10% 확률로 무작위 탐험 수행

3.2 주요 클래스

Bandit 클래스: 10개의 팔을 가진 슬롯 머신 환경을 시뮬레이션합니다. 각 팔은 무작위로 생성된 보상 확률을 가지고 있으며, play() 메서드를 통해 선택된 팔에 대한 보상(0 또는 1)을 반환합니다.

Agent 클래스: epsilon-greedy 전략을 사용하여 행동을 선택하는 에이전트입니다. 각 팔의 가치(Q값)를 추정하고, get_action()으로 행동을 선택하며, update()로 경험을 통해 학습합니다.

3.3 시뮬레이션 루프

이중 루프 구조로 되어 있습니다:

1. 외부 루프 (runs): 200번의 독립적인 실험 반복
 - 매 실험마다 새로운 Bandit과 Agent 생성
 - 누적 보상과 보상률 초기화
2. 내부 루프 (steps): 각 실험당 1000번의 스텝 수행
 - Agent가 행동 선택 (epsilon-greedy)
 - Bandit으로부터 보상 획득
 - Agent의 가치 추정 업데이트
 - 평균 보상률 계산: $\text{total_reward} / (\text{step} + 1)$

3.4 가치 함수 업데이트

```
self.Qs[action] += (reward - self.Qs[action]) / self.ns[action]
```

점진적 평균(Incremental Mean) 방식으로 Q값을 업데이트합니다. 이는 메모리 효율적이며 실시간 학습에 적합합니다. 새로운 보상이 들어올 때마다 기존 추정값과의 차이를 선택 횟수로 나누어 반영합니다.

4. 결과 분석

평균 보상률 그래프: 200번의 실험에 대한 평균을 계산하여 각 스텝별 평균 보상률의 변화를 시각화합니다. 그래프는 학습이 진행됨에 따라 보상률이 점차 증가하고 안정화되는 모습을 보여줍니다.

초기에는 탐험을 통해 좋은 팔을 찾아가는 과정에서 변동성이 크지만, 시간이 지남에 따라 최적의 팔에 수렴하면서 보상률이 안정화됩니다. 이는 epsilon-greedy 전략이 효과적으로 작동하고 있음을 보여줍니다.

5. 핵심 알고리즘 정리

단계	설명	수식
1. 초기화	Q값과 선택 횟수 초기화	$Q(a) = 0, N(a) = 0$
2. 행동 선택	ϵ 확률로 무작위, $1-\epsilon$ 확률로 최대 Q값	$a = \operatorname{argmax} Q(a)$
3. 보상 획득	선택한 팔에서 보상 받기	$r \in \{0, 1\}$
4. 가치 업데이트	점진적 평균으로 Q값 갱신	$Q(a) \leftarrow Q(a) + (r - Q(a))/N(a)$
5. 반복	충분히 학습될 때까지 2-4 반복	-

6. 학습 포인트

- 탐험-활용 딜레마: 새로운 정보를 얻기 위한 탐험과 현재 최선의 선택을 하는 활용 사이의 균형
- 점진적 학습: 모든 데이터를 저장하지 않고도 평균을 계산하는 효율적인 방법
- 통계적 검증: 여러 번의 독립적인 실험을 통해 결과의 신뢰도 향상
- 비정상성(Non-stationarity): 환경이 변하지 않는 정상 문제에 대한 기본적인 접근

7. 실행 방법

```
python bandit_avg.py
```

스크립트를 실행하면 200번의 실험이 자동으로 수행되고, 평균 보상률을 나타내는 그래프가 표시됩니다.